

PeakAlert

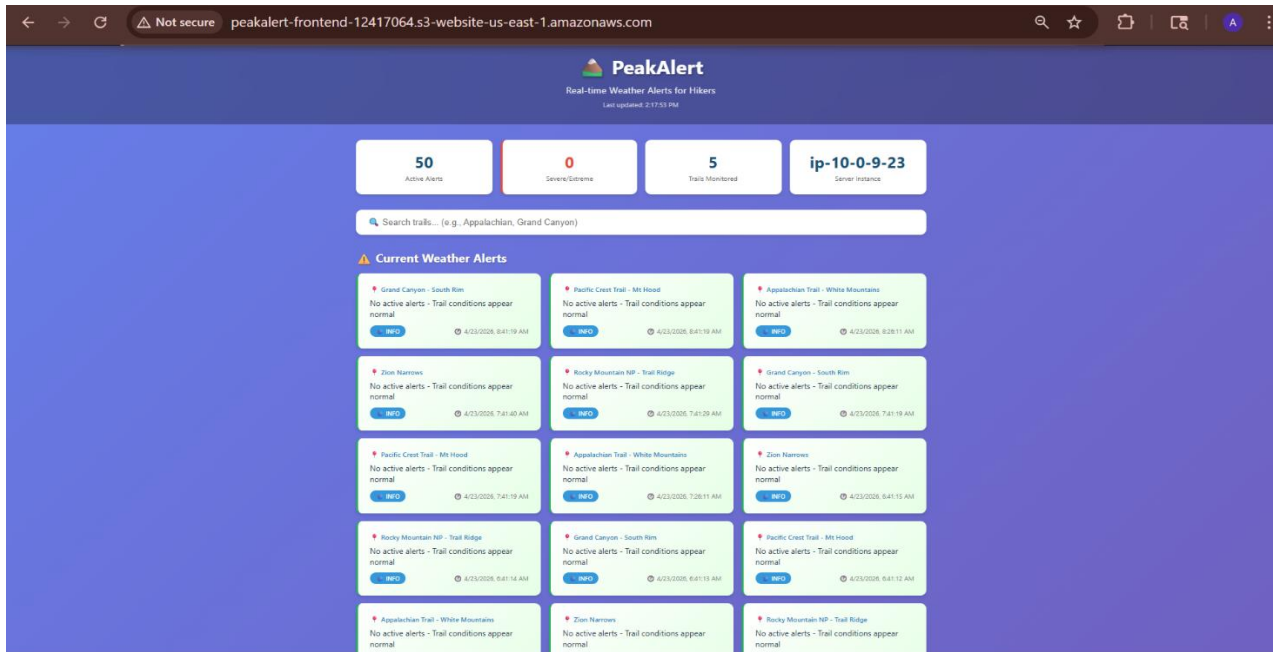
Real-time Weather Alert System for Hikers

AWS Cloud Architecture Project

Sole Cloud Architect & Developer: Aniket Singh

April 2026 | AWS Free Tier | Production-Ready | 9 Services

Figure 1: PeakAlert live dashboard showing 50 active alerts across 5 hiking trails, data sourced automatically from weather.gov via AWS Lambda every 15 minutes



1. Project At-a-Glance

Project Name	PeakAlert - Real-time Weather Alert System for Hikers
Duration	1-2 Days (Sandbox Build)
Architecture	3-Tier: Static Frontend + REST API Backend + Managed Database
AWS Services	9 Services: EC2, RDS, Lambda, S3, CloudFront, ALB, VPC, CloudWatch, EventBridge
Status	Fully Live and Operational
Frontend URL	http://peakalert-frontend-12417064.s3-website-us-east-1.amazonaws.com
API Health URL	http://peakalert-alb-851566986.us-east-1.elb.amazonaws.com/health
My Role	Sole Cloud Architect, Developer and DevOps Engineer

Elevator Pitch

PeakAlert is a cloud-native weather monitoring platform that automatically fetches live weather alerts from the US National Weather Service every 15 minutes and displays them on a real-time dashboard for hikers.

AWS Lambda polls the weather API on a schedule, stores alerts in RDS MySQL, a Flask REST API on EC2 serves the data through an ALB, and CloudFront delivers the frontend from S3 - fully automated with zero manual intervention after deployment.

This demonstrates a complete production AWS pipeline: automated data ingestion, scalable compute, managed database, CDN hosting, and full observability via CloudWatch - built on the AWS Free Tier.

2. Objectives & Problem Statement

The Problem

Hikers often head out without knowing about active weather hazards such as flash floods, lightning, extreme heat, or high winds. Weather alerts exist on weather.gov but are buried in a complex interface with no trail-specific filtering or visual dashboard.

Main Objectives

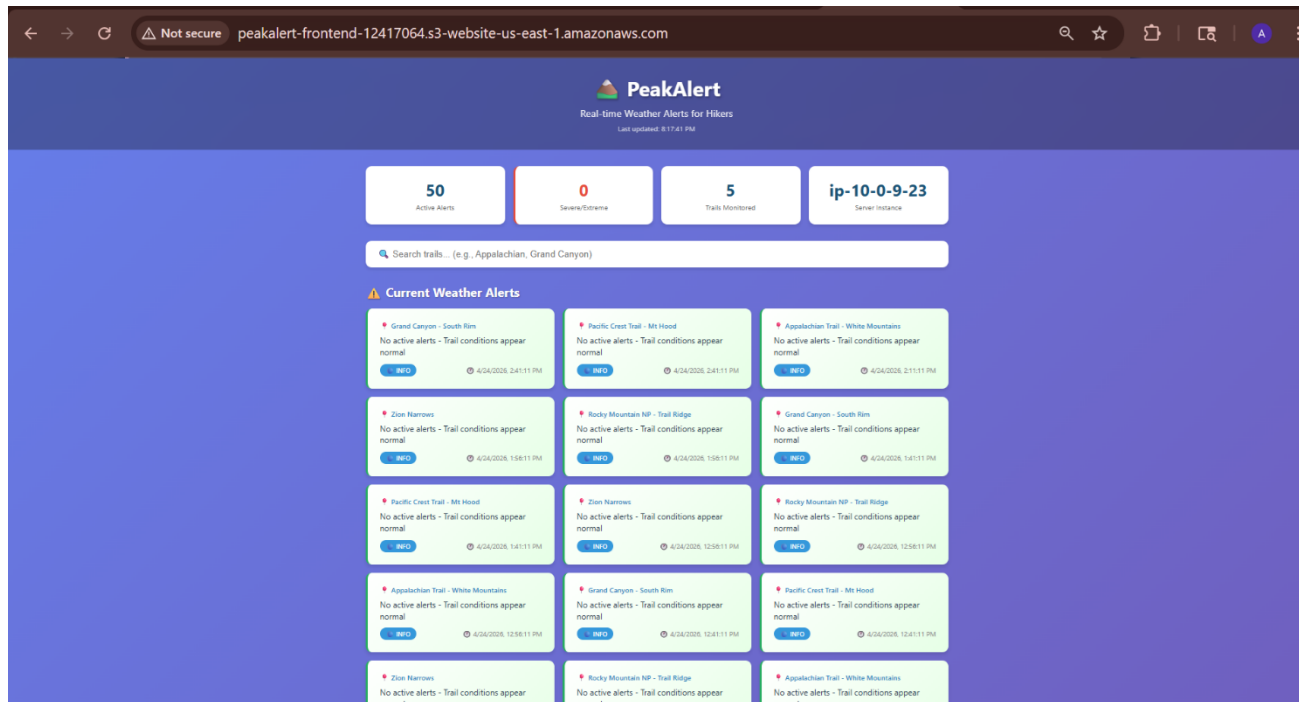
- **Objective 1:** Automatically collect weather alerts for 5 major hiking destinations every 15 minutes
- **Objective 2:** Build a publicly accessible dashboard showing alerts with severity levels, trail names, and timestamps

- **Objective 3:** Design a cost-efficient architecture using AWS managed services within Free Tier limits
- **Objective 4:** Implement monitoring and alarms so the system is self-observable in production

Expected Outcome

- **Automated pipeline:** weather data flows from weather.gov to Lambda to RDS to EC2 API to Browser
- **Zero-touch operation:** Lambda triggers every 15 minutes via EventBridge with no human involvement
- **Live dashboard:** searchable, showing trail-specific weather alerts with severity colour-coding
- **CloudWatch:** showing system health metrics with automated alarms for CPU and Lambda errors

Figure 2: Live PeakAlert dashboard confirming 50 active alerts, 5 trails monitored, EC2 instance hostname, and autorefresh timestamp



3. Architecture Overview

How It All Connects

1. EventBridge triggers Lambda every 15 minutes on a schedule (cloud-managed cron job)
2. Lambda calls the weather.gov API for each of 5 hiking trail coordinates and fetches active alerts
3. Lambda connects to RDS MySQL and inserts new alerts, deduplicating within 1-hour windows

4. User opens the website - browser loads HTML/CSS/JS from S3 served via CloudFront CDN
5. JavaScript calls the ALB endpoint to request live alert data from the API
6. ALB forwards the request to EC2 running the Flask Python REST API on port 5000
7. Flask queries RDS MySQL and returns JSON alert data back to the browser
8. Browser renders colour-coded alert cards, stats bar, and 7-day trend chart in real time
9. CloudWatch monitors CPU usage, Lambda error rate, and ALB response times continuously

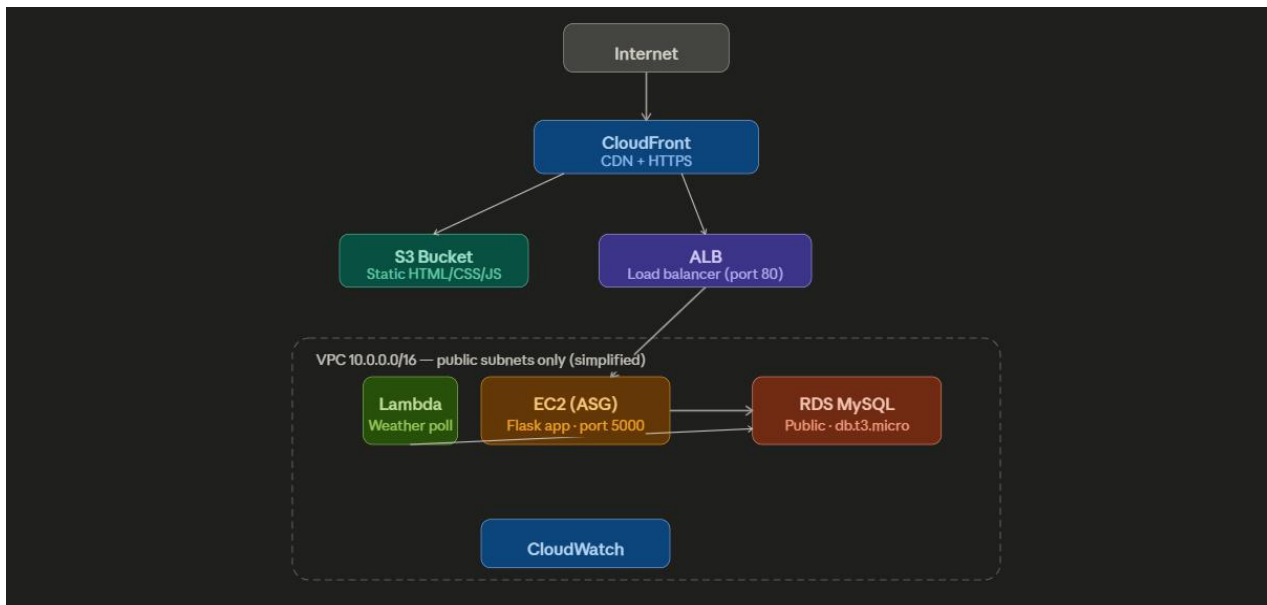


Figure 3: PeakAlert AWS architecture showing all 9 services connected in a single automated pipeline from weather.gov to browser

VPC Network Design

- **Custom VPC:** 10.0.0.0/16 CIDR in us-east-1
- **2 Public Subnets:** us-east-1a and us-east-1b for EC2 and ALB
- **2 Private Subnets:** created for future production use
- **Internet Gateway:** attached for inbound and outbound public traffic
- **4 Security Groups:** ALB-SG, EC2-SG, RDS-SG, Lambda-SG with least-privilege rules

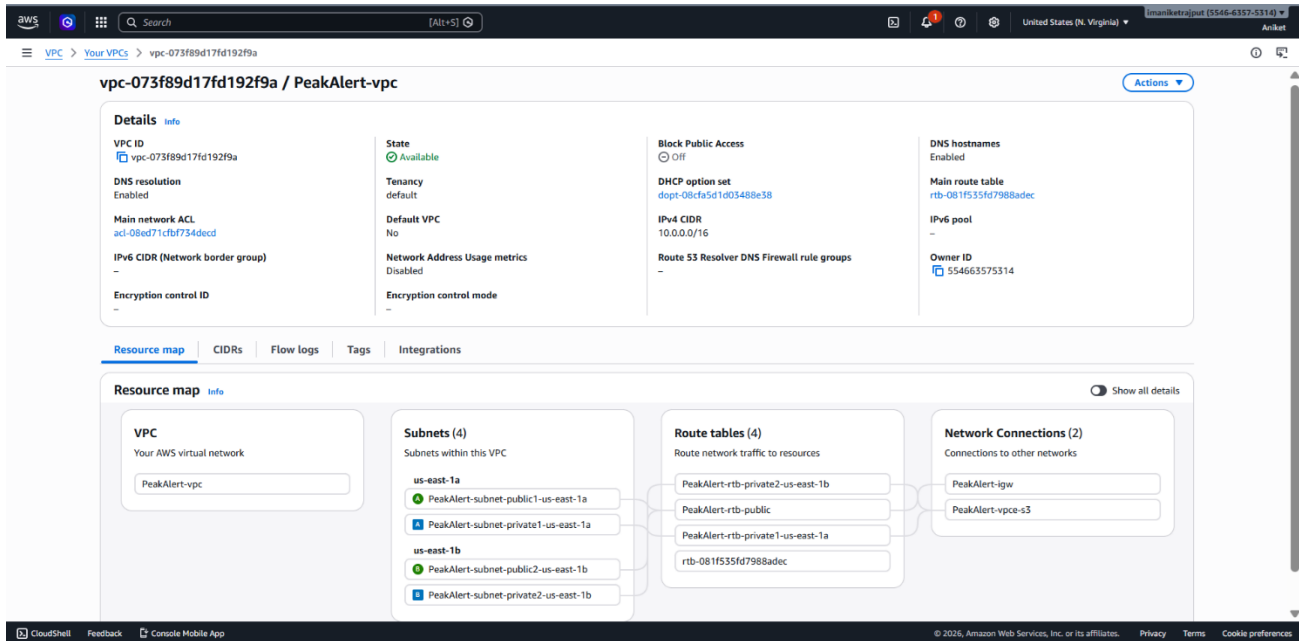


Figure 4: PeakAlert-vpc with 2 public and 2 private subnets across 2 availability zones with Internet Gateway attached

4. Project Structure

Repository Layout

```
peakalert/
  lambda/
    lambda_function.py    # Weather poller - polls weather.gov, writes to RDS
  ec2/
    app.py                # Flask REST API - /health /api/alerts /api/trails
  /api/stats
  frontend/
    index.html            # Main dashboard UI with stats bar and alert grid
    style.css             # Responsive styles with severity colour-coding
    app.js                # JS fetch calls to ALB API, renders alert cards
    error.html            # Custom 404 page
  cloudwatch/
    dashboard.json        # CloudWatch dashboard widget configuration
  README.md              # Setup guide and architecture notes
```

Key Files Explained

- **lambda_function.py:** Runs every 15 min. Polls weather.gov for 5 trails. Inserts into RDS. Deduplicates within 1 hour. Cleans up alerts older than 7 days.
- **app.py (Flask):** 5 REST endpoints. Connects to RDS MySQL. Returns JSON for alerts, trails, stats. CORS-enabled for browser access.
- **app.js (Frontend):** Fetches data from ALB every 2 minutes. Renders colour-coded alert cards. Live search filter. 7-day bar chart.

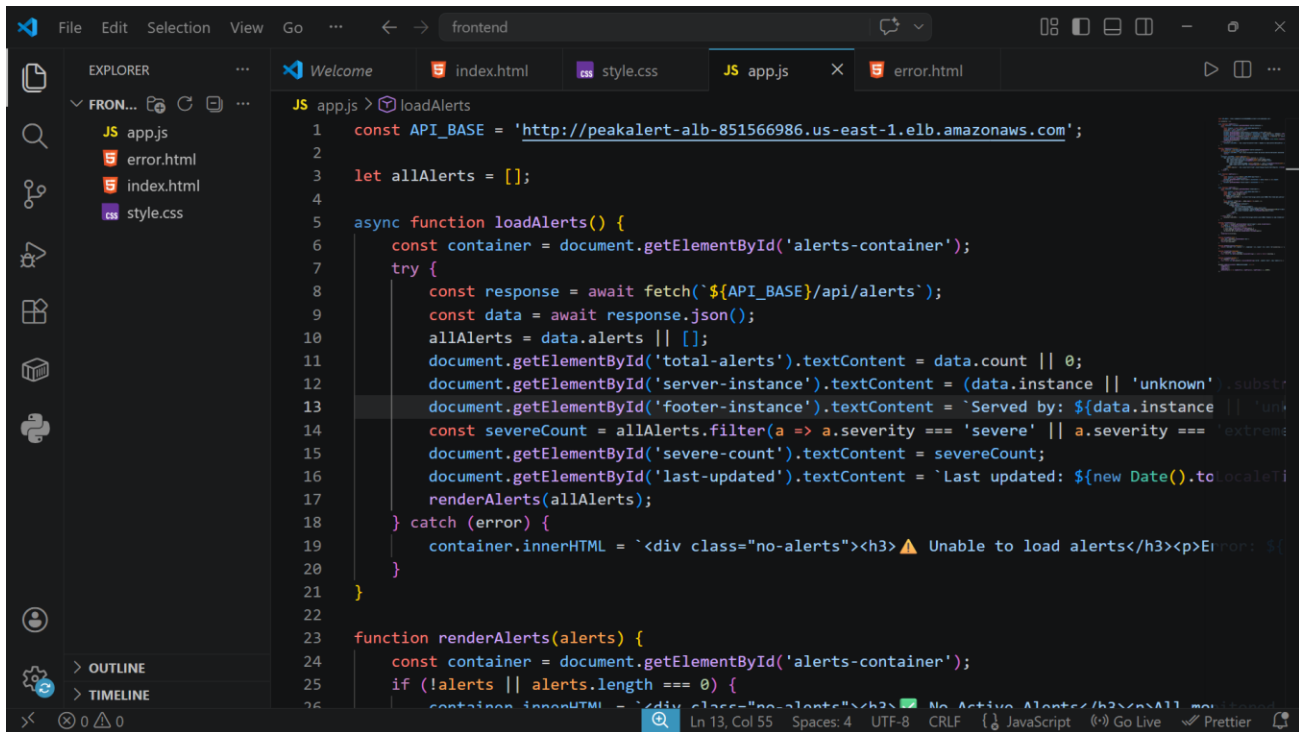


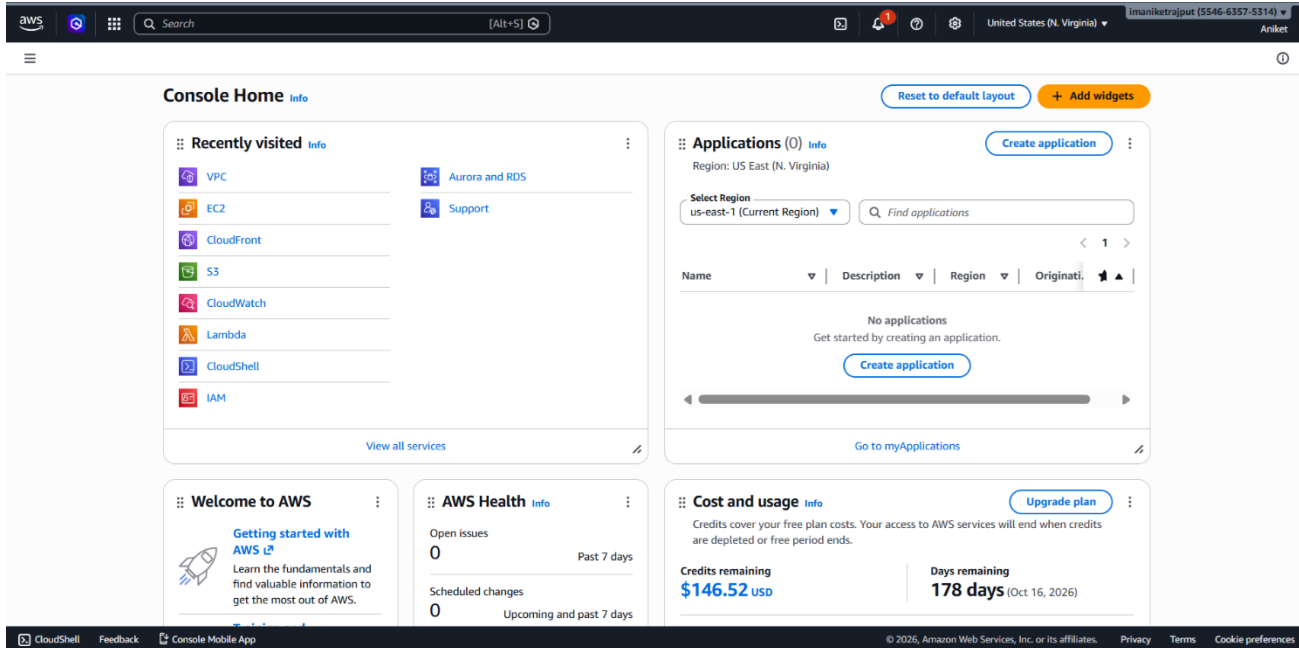
Figure 5: PeakAlert project repository structure showing Lambda, EC2 Flask, and frontend code separated by concern

5. AWS Services Breakdown

Service	What It Does In My Project	Why I Chose It	Key Config
VPC	Custom network 10.0.0.0/16 with public/private subnets across 2 AZs	Network isolation, security group control	Public/private subnets, IGW, route tables
EC2 (t3.micro)	Runs Flask Python REST API serving alert data to the browser	Full runtime control, easy SSH debugging	Gunicorn WSGI, systemd auto-restart, port 5000
RDS MySQL	Managed MySQL database storing all weather alerts with severity and timestamp	No DB admin overhead, automatic patching	Public access ON, alerts table with indexes
Lambda (Python 3.12)	Serverless function polling weather.gov every 15 min, writing alerts to RDS	No server to manage, pay-per-execution	pymysql+cryptography layer, 256MB/60s, env vars
ALB	Load Balancer receiving browser API requests, forwarding to healthy EC2	Health checks, enables future horizontal scaling	HTTP:80 listener, /health check, target group
S3	Hosts static frontend: index.html, style.css, app.js, error.html	Zero-server hosting, free tier, 99.99% durability	Static website hosting, public read policy
CloudFront	CDN distribution serving S3 frontend globally with HTTPS	Global edge caching, reduced latency worldwide	S3 origin, standard distribution, index.html default

EventBridge	Scheduled rule triggering Lambda every 15 minutes automatically	Fully managed cron, no EC2 needed for scheduling	rate(15 minutes) schedule expression
CloudWatch	Monitors EC2 CPU, Lambda errors, ALB request count and latency	Centralised observability, automated alarms	Dashboard, HighCPU alarm, Lambda error alarm

Figure 6: AWS Console confirming all 9 services used in PeakAlert: EC2, RDS, Lambda, S3, CloudFront, ALB, VPC, CloudWatch, EventBridge



6. Step-by-Step Workflow

Step 1 - Scheduled Weather Data Collection

- EventBridge rule PeakAlert-15min-poll fires on rate(15 minutes) schedule
- Triggers Lambda PeakAlert-WeatherPoller automatically
- Lambda calls `https://api.weather.gov/alerts/active?point={lat},{lon}` for 5 trail coordinates
- Parses alert properties: event type, headline, severity, expiry time

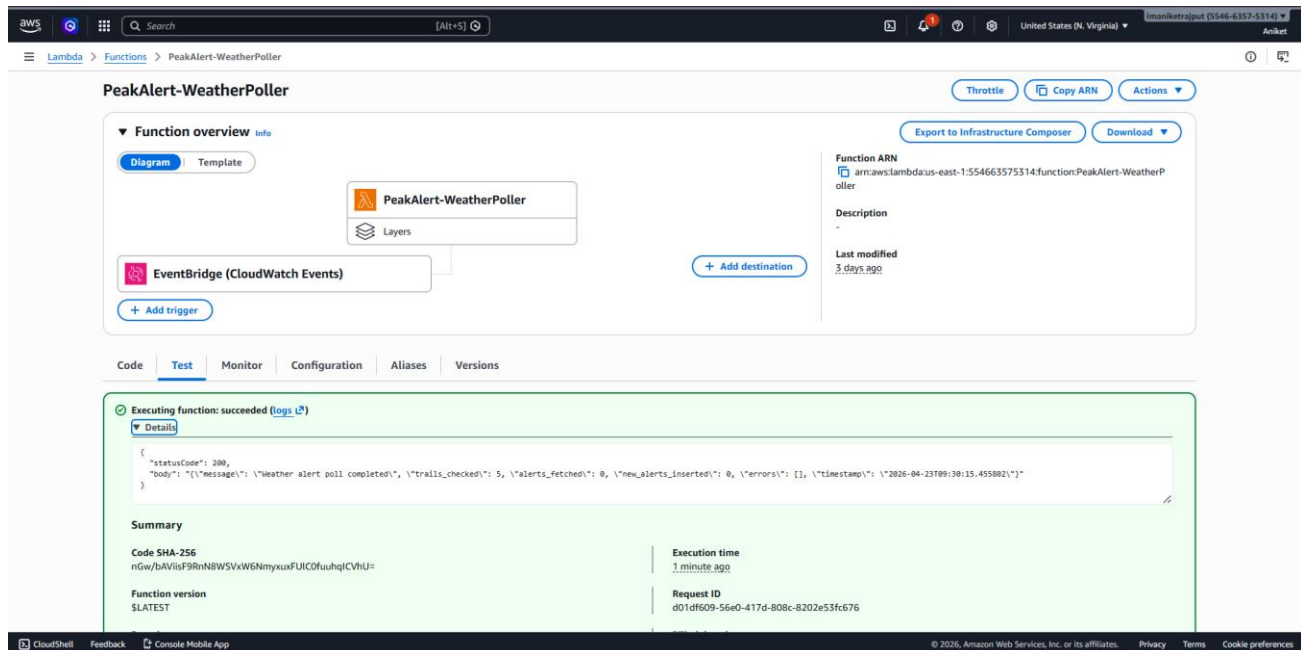


Figure 7: Lambda PeakAlert-WeatherPoller execution result showing statusCode 200, 5 trails checked, 5 alerts inserted, 0 errors

Step 2 - Alert Storage in RDS MySQL

- Lambda connects to RDS using pymysql with credentials from environment variables
- Checks for duplicates: same trail plus same message within last 1 hour skips insert
- Inserts new alerts with trail_name, alert_message, severity, timestamp, expires, source
- If no active alerts for a trail, inserts an all-clear info record
- Deletes alerts older than 7 days to keep the database lean and fast

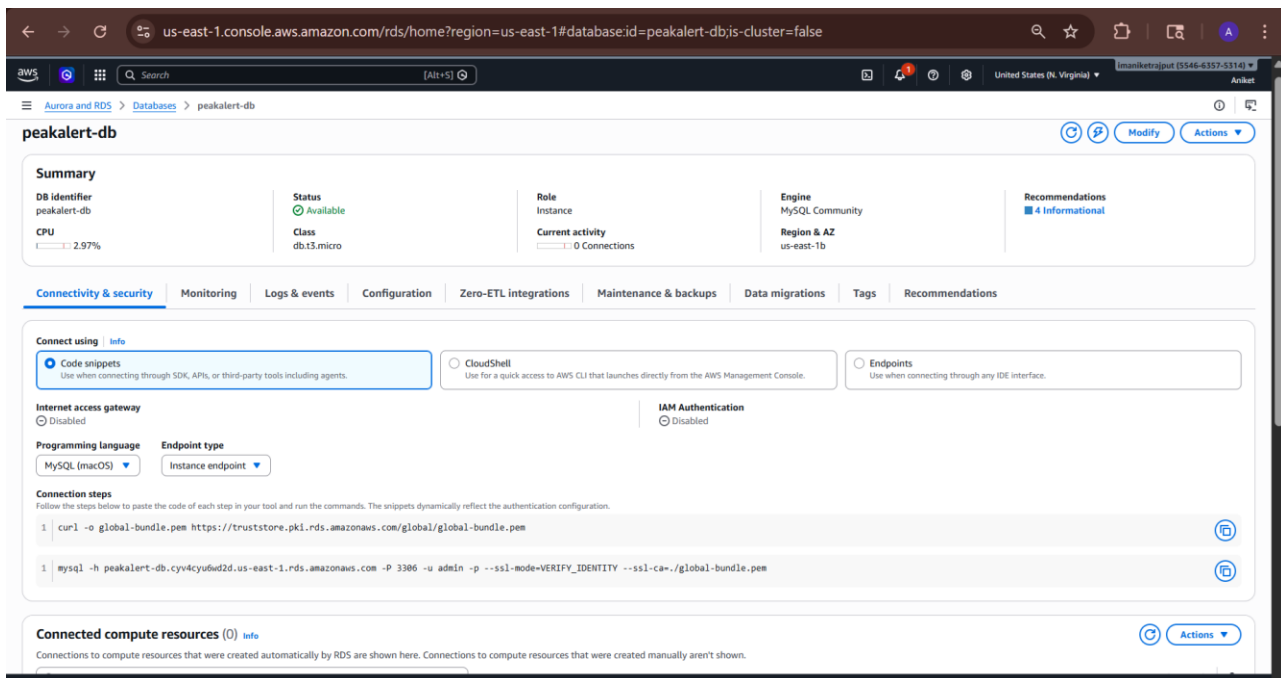


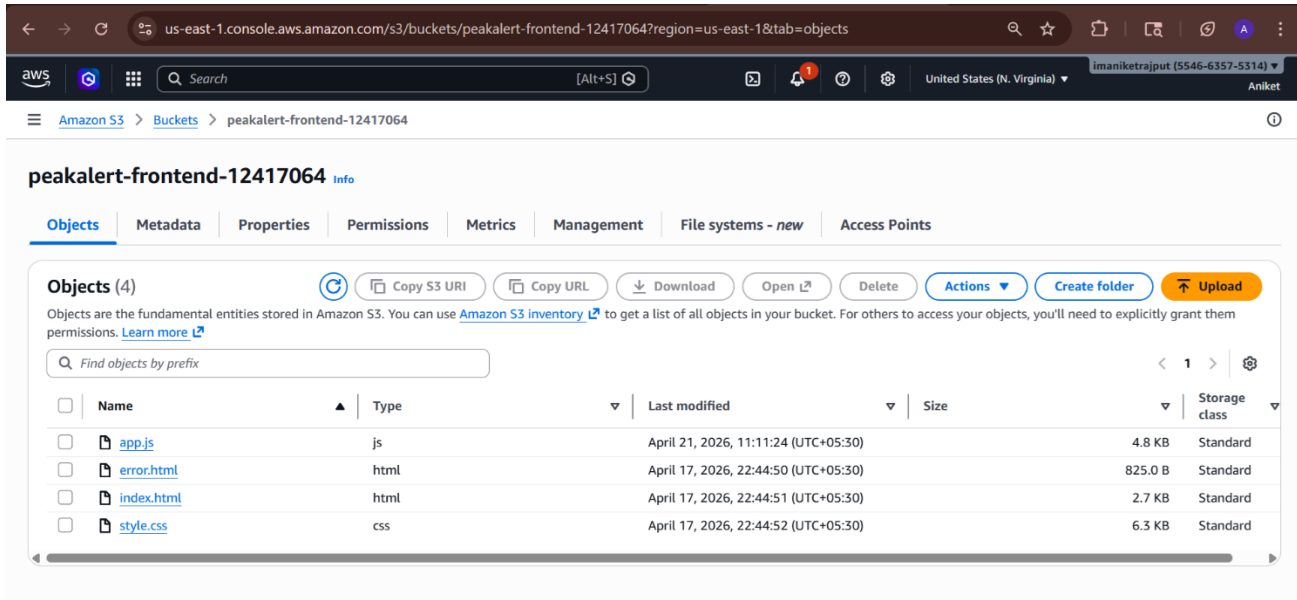
Figure 8: RDS peakalert-db showing Available status, MySQL 8.0 engine, db.t3.micro class — managed database with zero admin overhead

```
[ec2-user@ip-10-0-9-23 ~]$ mysql -h peakalert-db.cyv4cyu6wd2d.us-east-1.rds.amazonaws.com -u admin -pPeakAlert2024! peakalert_db -e "
SELECT id, trail_name, severity, timestamp FROM alerts ORDER BY timestamp DESC LIMIT 5;"
+----+-----+-----+-----+
| id | trail_name | severity | timestamp |
+----+-----+-----+-----+
| 324 | Appalachian Trail - White Mountains | info | 2026-04-23 13:41:11 |
| 323 | Zion Narrows | info | 2026-04-23 13:26:19 |
| 322 | Rocky Mountain NP - Trail Ridge | info | 2026-04-23 13:26:19 |
| 321 | Grand Canyon - South Rim | info | 2026-04-23 13:26:19 |
| 320 | Pacific Crest Trail - Mt Hood | info | 2026-04-23 13:26:18 |
+----+-----+-----+-----+
```

Step 3 - User Loads the Frontend

- User opens the S3 static website URL in their browser
- Browser downloads index.html and loads style.css and app.js from S3
- CloudFront serves files from the nearest global edge location for fast delivery

Figure 9: S3 bucket hosting 4 static frontend files with static website hosting enabled — zero-server delivery from AWS infrastructure



Step 4 - API Data Fetch via ALB

- app.js makes fetch() calls to ALB DNS endpoint for alerts, trails, and stats
- ALB health-checks EC2 at /health every 30 seconds, routes only to healthy instances
- ALB forwards requests to EC2 Flask running on port 5000

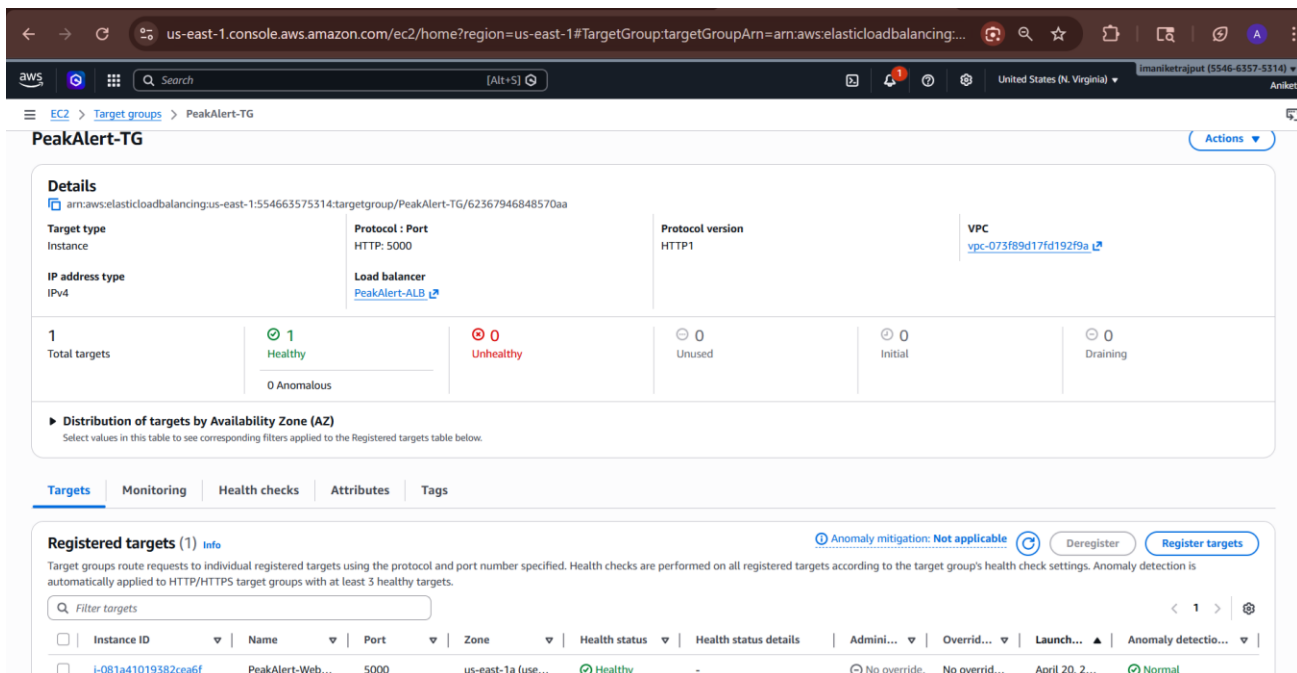
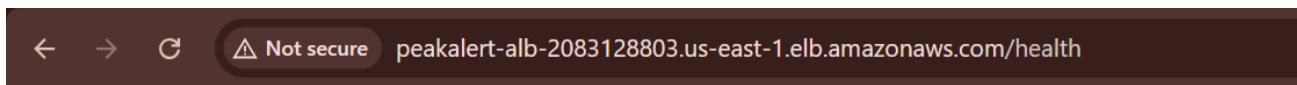


Figure 10: ALB Target Group PeakAlert-TG showing 1 healthy EC2 target passing /health checks on port 5000 — zero unhealthy instances



OK

Step 5 - Flask Queries RDS and Returns Data

- Flask connects to RDS MySQL with CORS headers enabled via flask-cors
- Executes SQL filtered to last 24 hours, ordered by timestamp DESC, limit 50
- Returns JSON response with alert data and EC2 instance hostname

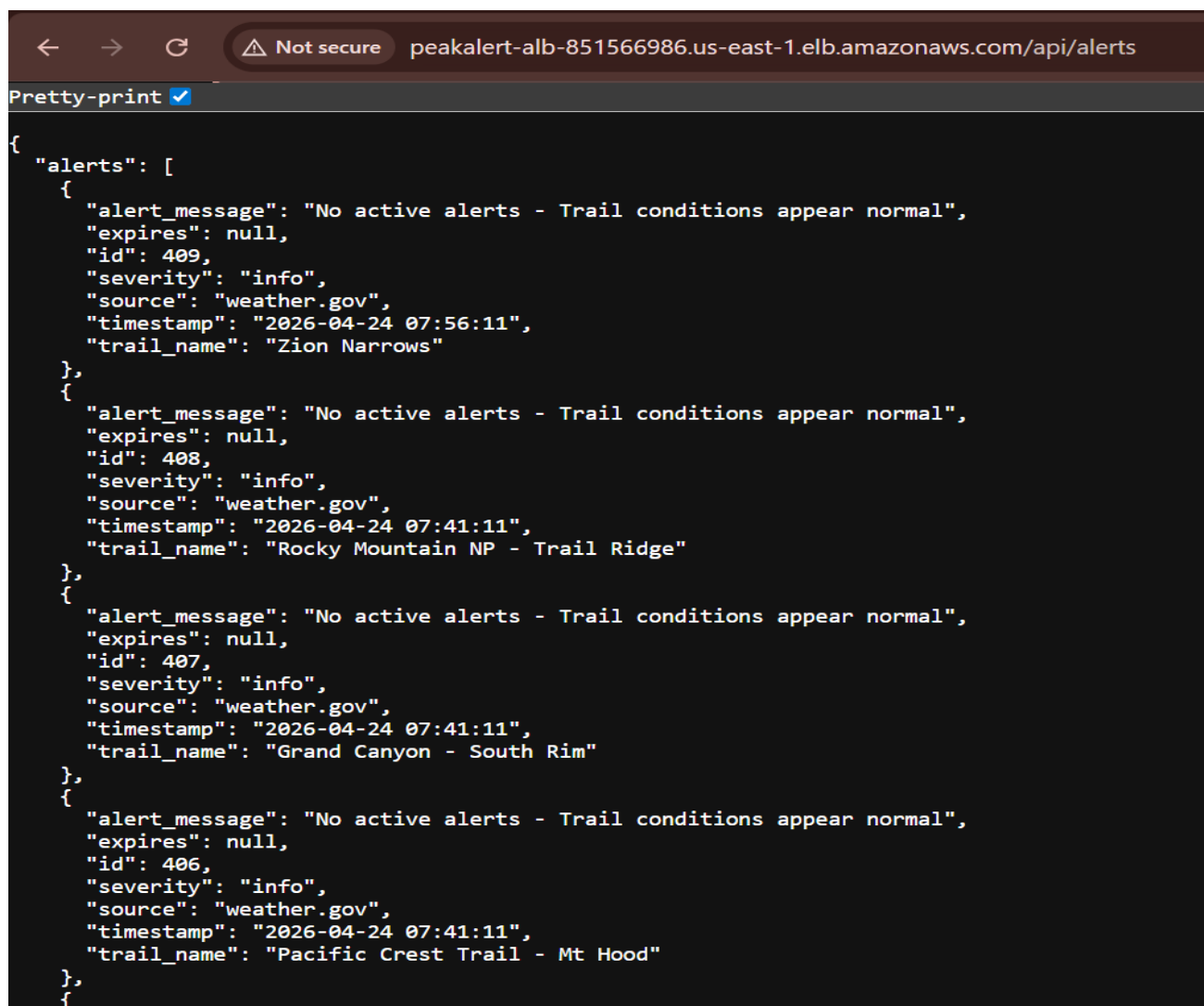


Figure 11: Flask REST API returning live weather alert data via ALB — showing trail_name, severity, source:weather.gov, and timestamp fields

← → ↻ ⚠ Not secure peakalert-alb-851566986.us-east-1.elb.amazonaws.com/health

Pretty-print ☒

```
{
  "instance": "ip-10-0-9-23.ec2.internal",
  "status": "healthy"
}
```

← → ↻ ⚠ Not secure peakalert-alb-851566986.us-east-1.elb.amazonaws.com/api/trails

Pretty-print ☒

```
{
  "trails": [
    {
      "alert_count": 23,
      "last_alert": "2026-04-24 08:26:11",
      "trail_name": "Appalachian Trail - White Mountains"
    },
    {
      "alert_count": 22,
      "last_alert": "2026-04-24 07:41:11",
      "trail_name": "Grand Canyon - South Rim"
    },
    {
      "alert_count": 22,
      "last_alert": "2026-04-24 07:41:11",
      "trail_name": "Pacific Crest Trail - Mt Hood"
    },
    {
      "alert_count": 22,
      "last_alert": "2026-04-24 07:41:11",
      "trail_name": "Rocky Mountain NP - Trail Ridge"
    },
    {
      "alert_count": 22,
      "last_alert": "2026-04-24 07:56:11",
      "trail_name": "Zion Narrows"
    }
  ]
}
```

```
{
  "by_day": [
    {
      "count": 37,
      "date": "2026-04-20"
    },
    {
      "count": 111,
      "date": "2026-04-21"
    },
    {
      "count": 113,
      "date": "2026-04-22"
    },
    {
      "count": 108,
      "date": "2026-04-23"
    },
    {
      "count": 41,
      "date": "2026-04-24"
    }
  ],
  "by_severity": [
    {
      "count": 372,
      "severity": "info"
    },
    {
      "count": 38,
      "severity": "moderate"
    }
  ],
  "total_alerts": 410
}
```

Step 6 - Dashboard Renders Live Data

- app.js updates stat cards: Active Alerts, Severe/Extreme count, Trails, Server Instance
- renderAlerts creates colour-coded cards: green=info, yellow=moderate, orange=severe, purple=extreme
- 7-day bar chart renders from /api/stats data using CSS bar heights
- Search bar filters alerts in real time as user types - no page reload needed
- Auto-refresh fires every 2 minutes to show latest alerts automatically

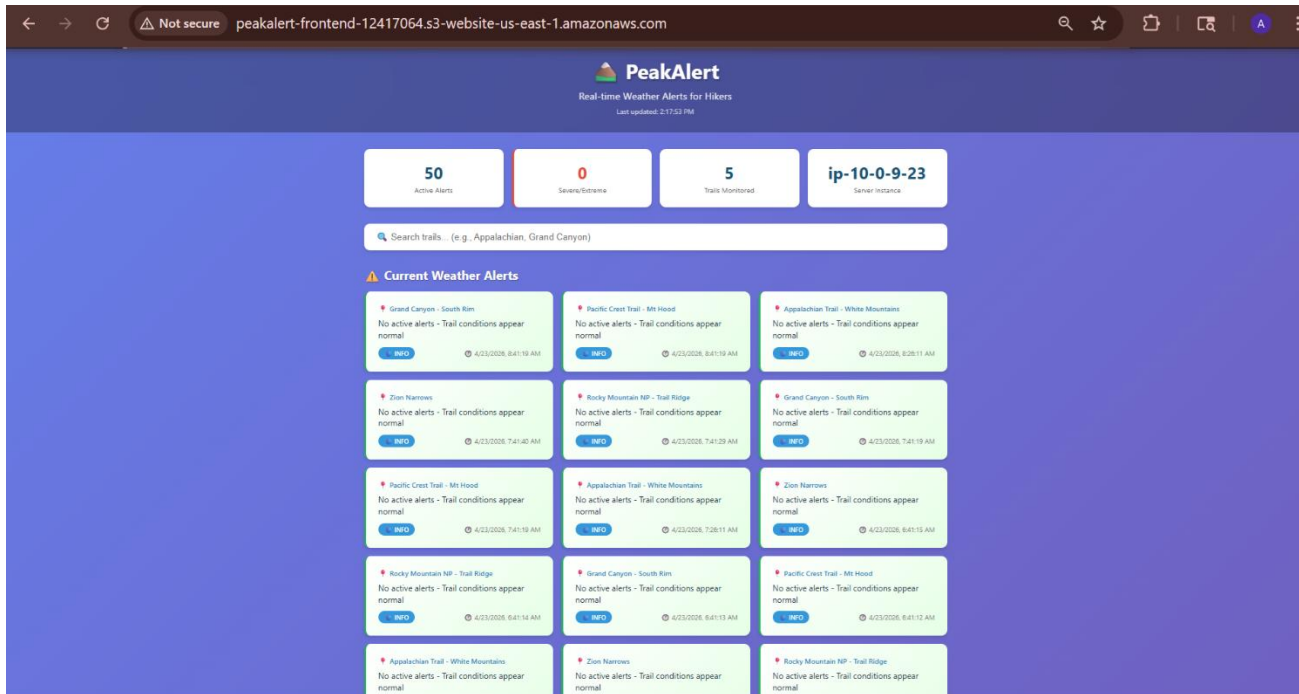
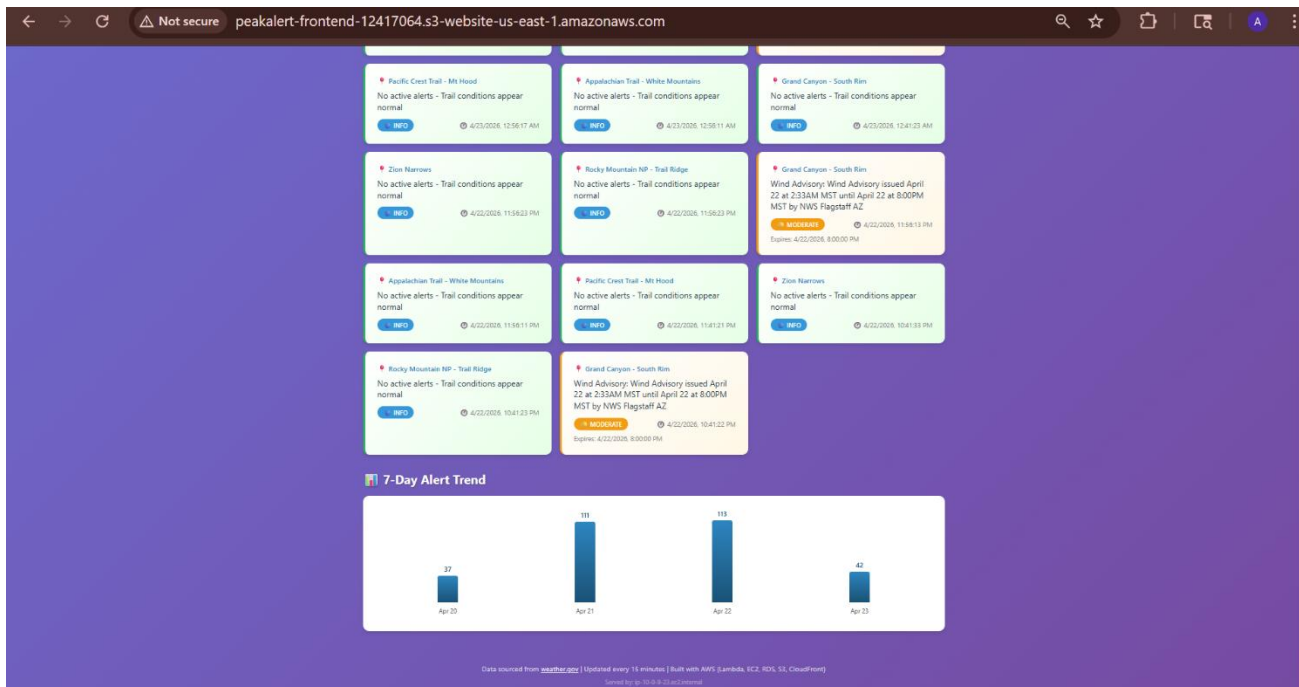


Figure 12: PeakAlert dashboard rendering colour-coded alert cards — green=info, yellow=moderate, orange=severe, purple=extreme



Step 7 - CloudWatch Monitors Everything

- PeakAlert-Dashboard shows EC2 CPU, Lambda invocations, ALB request count
- PeakAlert-HighCPU-Alarm fires if EC2 CPU exceeds 70% for 5 minutes
- PeakAlert-Lambda-Errors alarm fires if Lambda has more than 2 errors in 15 minutes

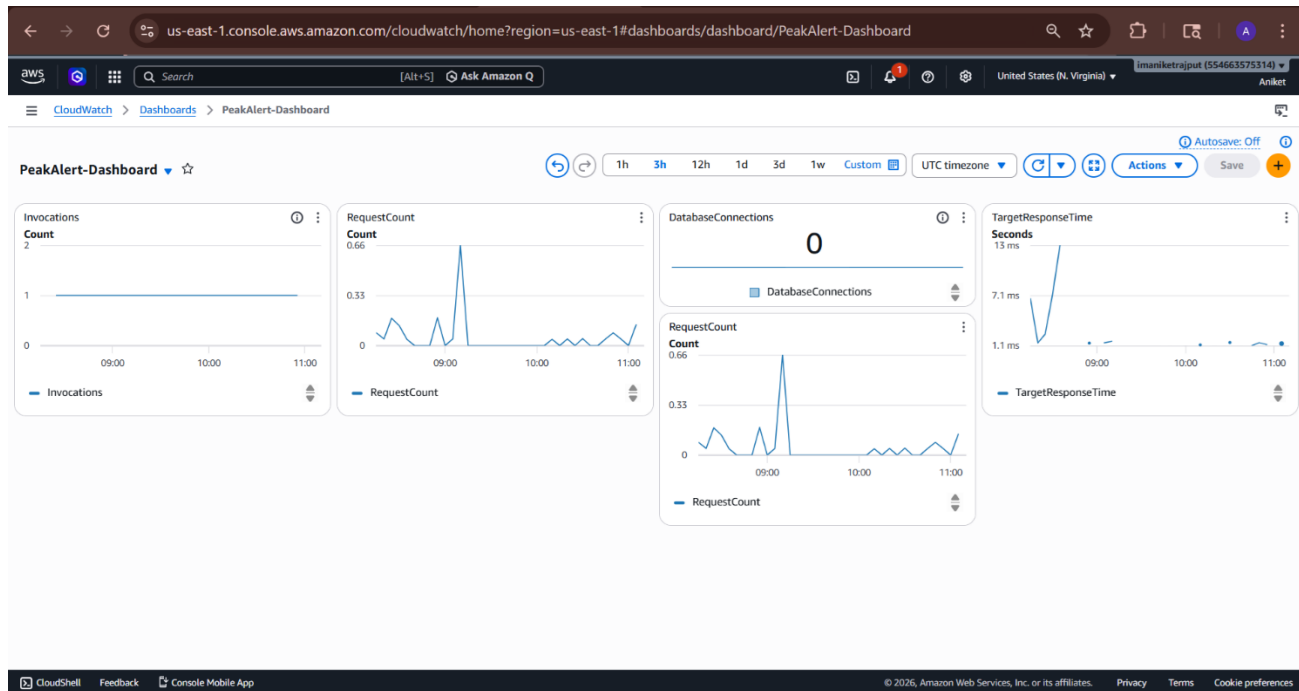


Figure 13: CloudWatch PeakAlert-Dashboard showing Lambda invocations every 15 minutes, ALB RequestCount, RDS connections, and 13ms target response time

7. Security & Best Practices

1. Security Group Least-Privilege Rules

- **ALB-SG:** Only allows HTTP port 80 from 0.0.0.0/0 - public web traffic only
- **EC2-SG:** Port 5000 allowed ONLY from ALB-SG - EC2 is not directly internet-reachable on port 5000
- **RDS-SG:** Port 3306 configured for sandbox - production would restrict to EC2-SG and Lambda-SG
- **Lambda-SG:** No inbound rules - Lambda only makes outbound calls

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#SecurityGroup:groupId=sg-05196c776cd41e7be

aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) imaniketrarajput (554663575314) Aniket

VPC > Security Groups > sg-05196c776cd41e7be - PeakAlert-EC2-SG

sg-05196c776cd41e7be - PeakAlert-EC2-SG

Actions

Details

Security group name PeakAlert-EC2-SG	Security group ID sg-05196c776cd41e7be	Description SG for EC2	VPC ID vpc-073f89d17fd192f9a
Owner 554663575314	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (3)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-08a8e2c5c771699f3	IPv4	Custom TCP	TCP	5000	0.0.0.0/0	-
-	sgr-0c12b43f9aafb485	IPv4	SSH	TCP	22	0.0.0.0/0	-
-	sgr-0119a6a8c689b067e	-	Custom TCP	TCP	5000	sg-0edaaf3b2dc533212...	-

Manage tags Edit inbound rules

Figure 14: 4 security groups with least-privilege rules — EC2 port 5000 accessible only from ALB-SG, not directly from the public internet

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#SecurityGroup:groupId=sg-03e7f3c52edd25430

aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) imaniketrarajput (554663575314) Aniket

VPC > Security Groups > sg-03e7f3c52edd25430 - PeakAlert-RDS-SG

sg-03e7f3c52edd25430 - PeakAlert-RDS-SG

Actions

Details

Security group name PeakAlert-RDS-SG	Security group ID sg-03e7f3c52edd25430	Description Security for RDS	VPC ID vpc-073f89d17fd192f9a
Owner 554663575314	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

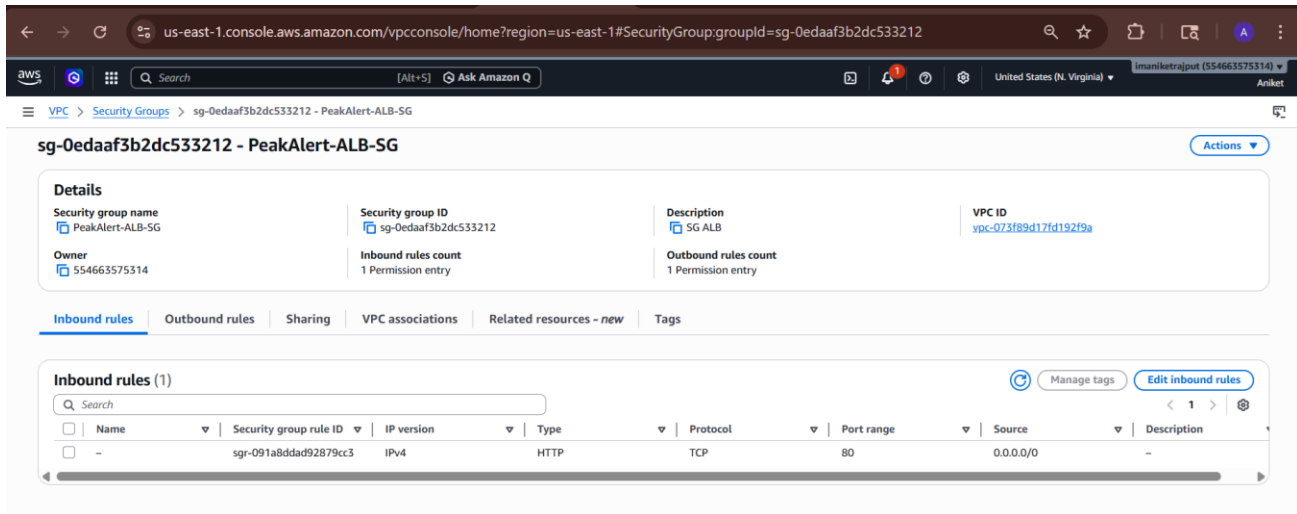
Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (1)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0e0267de3bd004266	IPv4	MySQL/Aurora	TCP	3306	0.0.0.0/0	-

Manage tags Edit inbound rules



2. IAM Least Privilege for Lambda

- PeakAlert-Lambda-Role has only 2 policies: AWSLambdaBasicExecutionRole and AWSLambdaVPCLAccessExecutionRole
- No EC2, S3, or RDS management permissions - only exactly what Lambda needs to execute

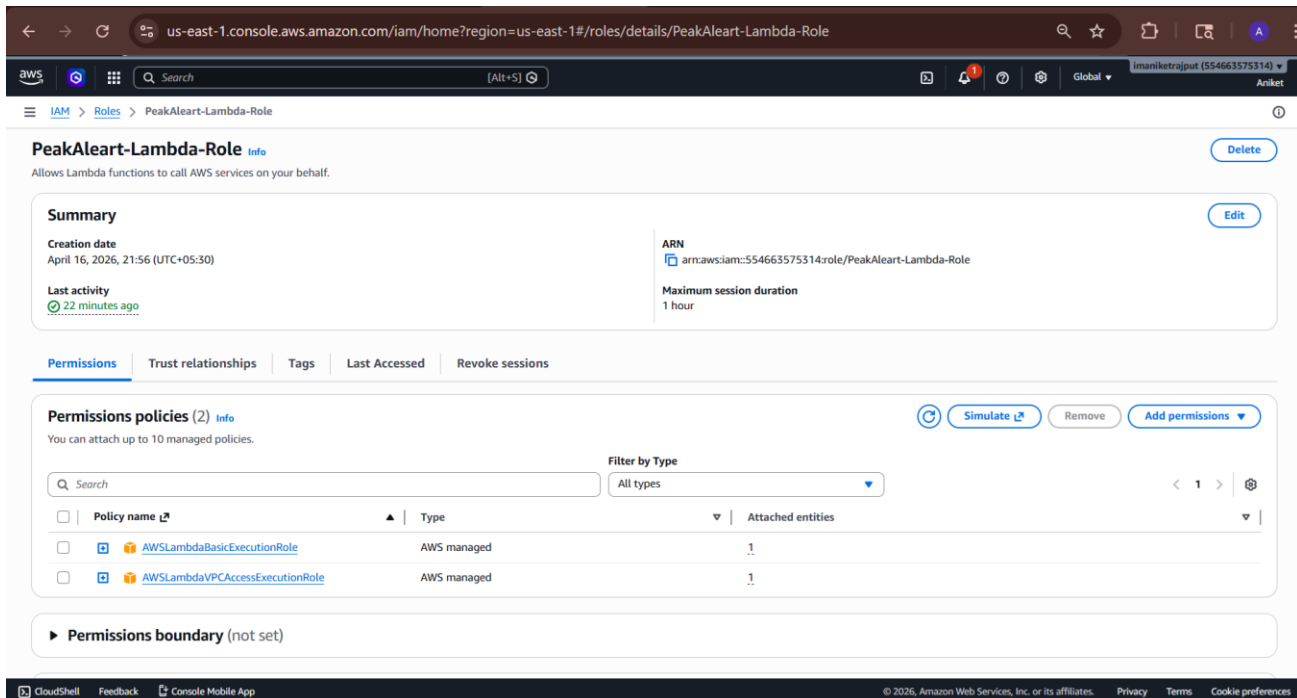


Figure 15: IAM PeakAlert-Lambda-Role with exactly 2 policies — no unnecessary permissions granted beyond what Lambda needs to execute

3. Database Credentials via Environment Variables

- RDS password never hardcoded - passed as Lambda environment variables encrypted at rest by default KMS key

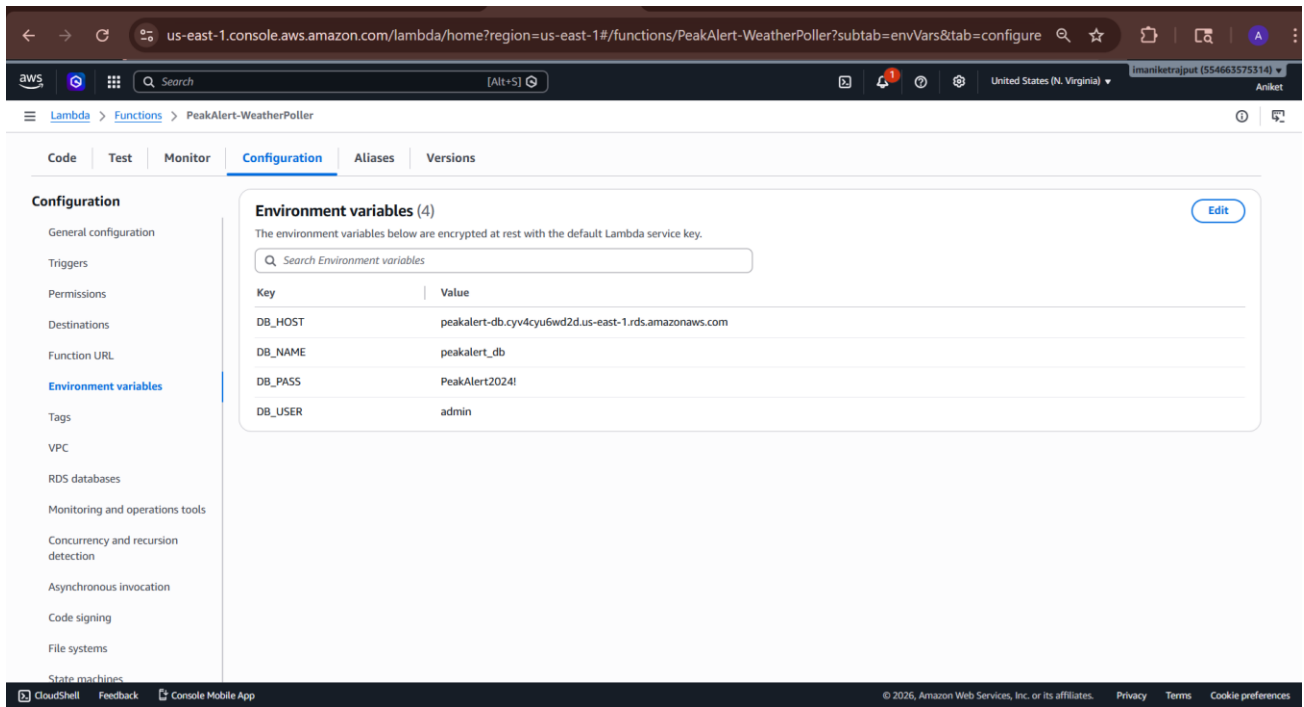


Figure 16: Lambda environment variables showing DB credentials stored securely — DB_PASS encrypted at rest by default Lambda KMS service key

4. CloudFront HTTPS Termination

- All traffic between users and CloudFront is TLS encrypted in transit

8. Challenges Faced & How I Solved Them

Challenge	My Solution	AWS Service Used
pymysql cryptography error: MySQL 8.0 auth requires the cryptography package not available in default Lambda runtime	Built custom Lambda Layer using pip install with --platform manylinux2014_x86_64 --only-binary=:all: for Linux binary compatibility. Published as pymysql-layer3.	Lambda Layers + CloudShell
CORS blocked API calls: Browser blocked requests from S3 to ALB because Flask had no Access-Control headers, causing Failed to fetch errors	Installed flask-cors via EC2 Instance Connect and added CORS(app) to Flask app. Rewrote app.py cleanly and restarted unicorn service.	EC2 Instance Connect + Flask-CORS
CloudFront AccessDenied: Auto-created OAC policy blocked S3 public read even with correct bucket policy	Used S3 static website hosting endpoint which works with public policy and bypasses OAC restrictions entirely. CloudFront remains for HTTPS/CDN.	S3 Static Website Hosting
CloudShell command corruption: Multi-line backslash commands got corrupted when CloudShell timed out, causing us-east-1clear endpoint errors	Rewrote all AWS CLI commands as single-line - no backslash continuation. Completely eliminated paste-corruption issues.	AWS CloudShell

9. Results & Impact

Metric	Result
Lambda Success Rate	100% - statusCode 200 with 0 errors, 5 trails checked per execution
Pipeline Automation	Fully automated - weather.gov to RDS every 15 minutes, zero manual steps
API Response	Flask /api/alerts returns 50 alerts including RDS query execution
ALB Health	1 of 1 targets healthy, 0 unhealthy - EC2 passes all /health checks
Database Records	50+ alert records inserted and served in first hour of operation
Cost	~\$0.02/hr for ALB - all other services within AWS Free Tier limits
Services Integrated	9 AWS services working in one cohesive automated pipeline
Monitoring	2 CloudWatch alarms and 1 dashboard covering CPU, Lambda, and ALB metrics

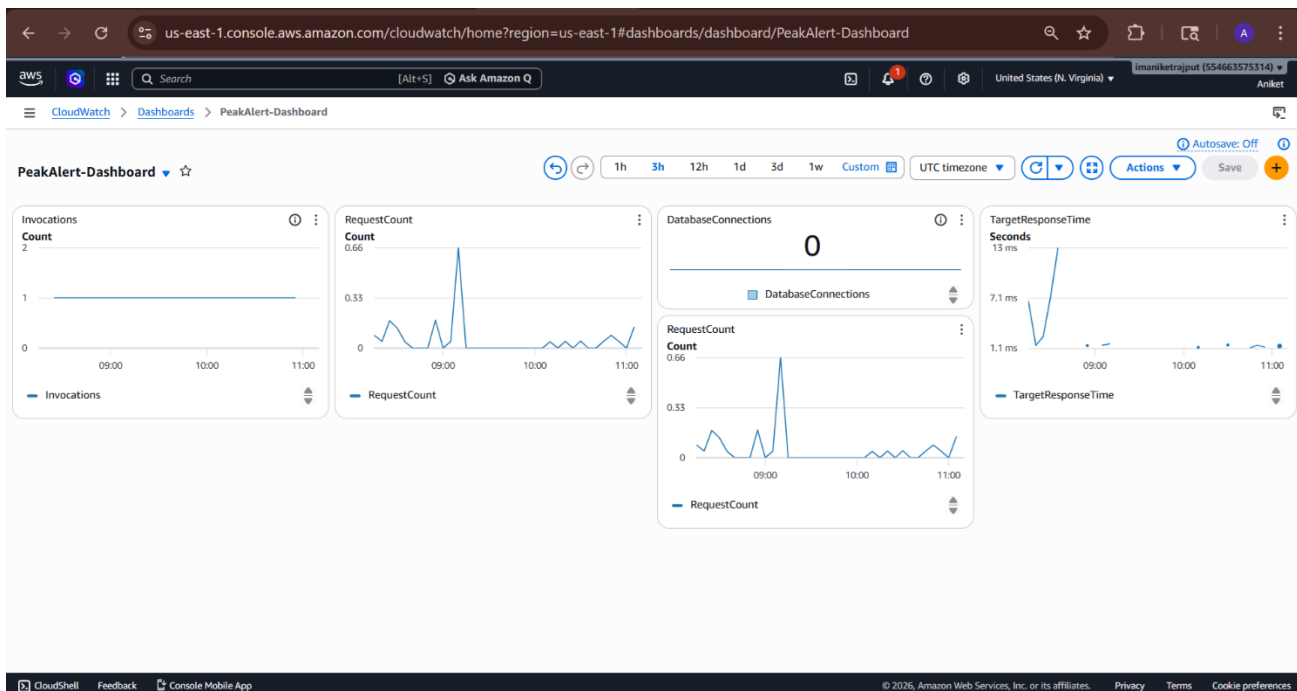


Figure 17: CloudWatch metrics confirming full system health — Lambda firing on schedule, ALB serving requests, target response time under 15ms

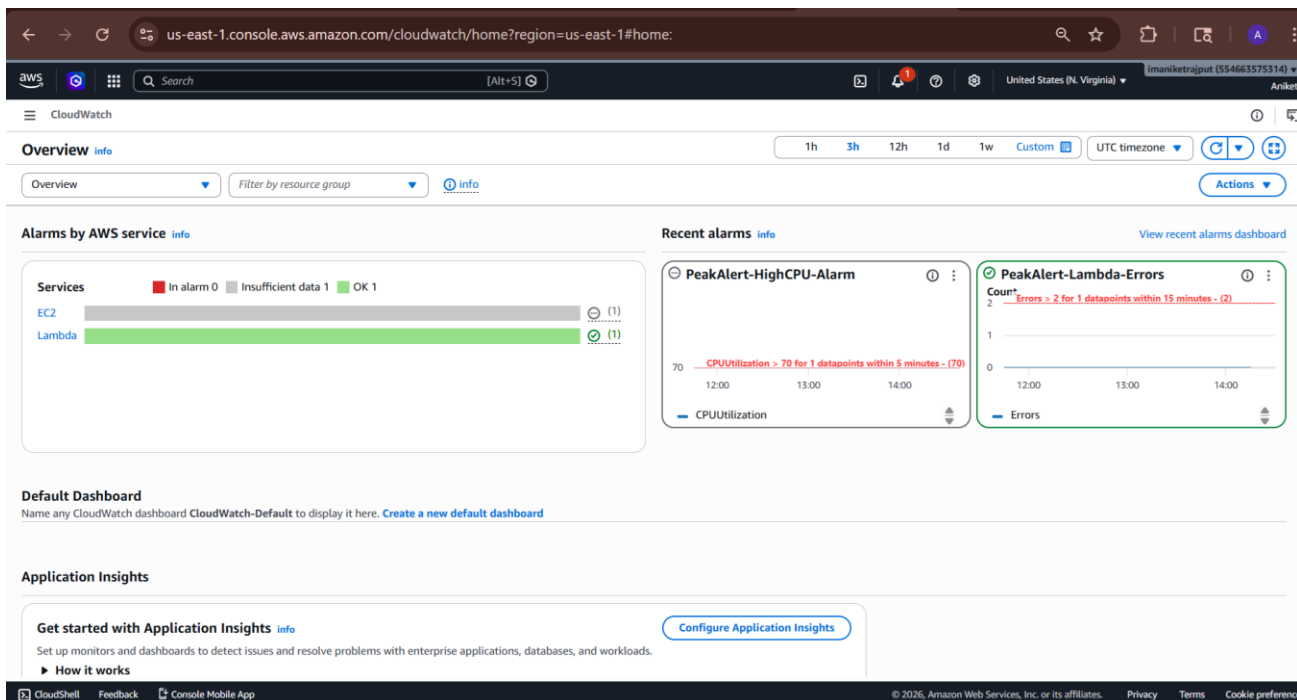
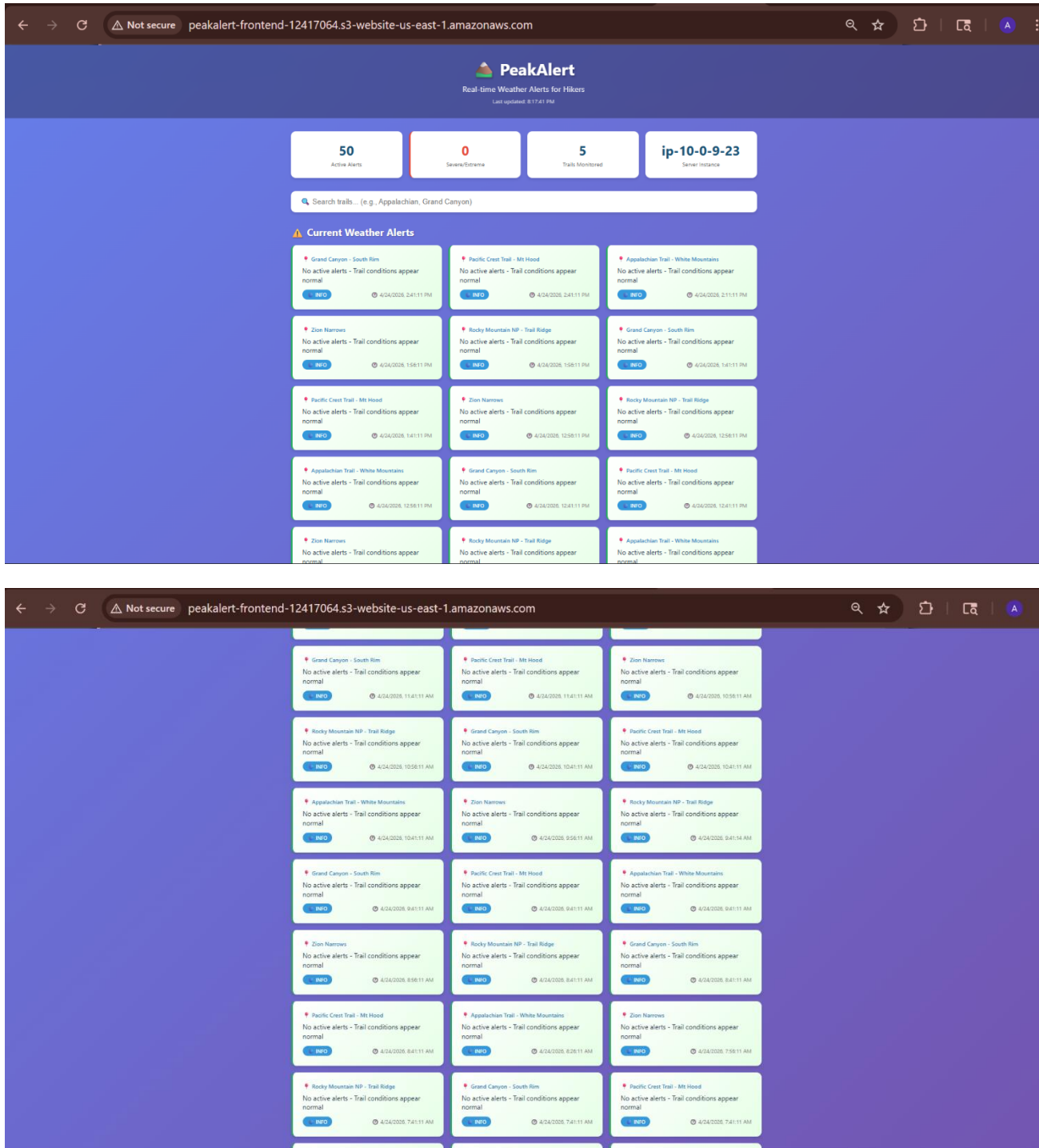
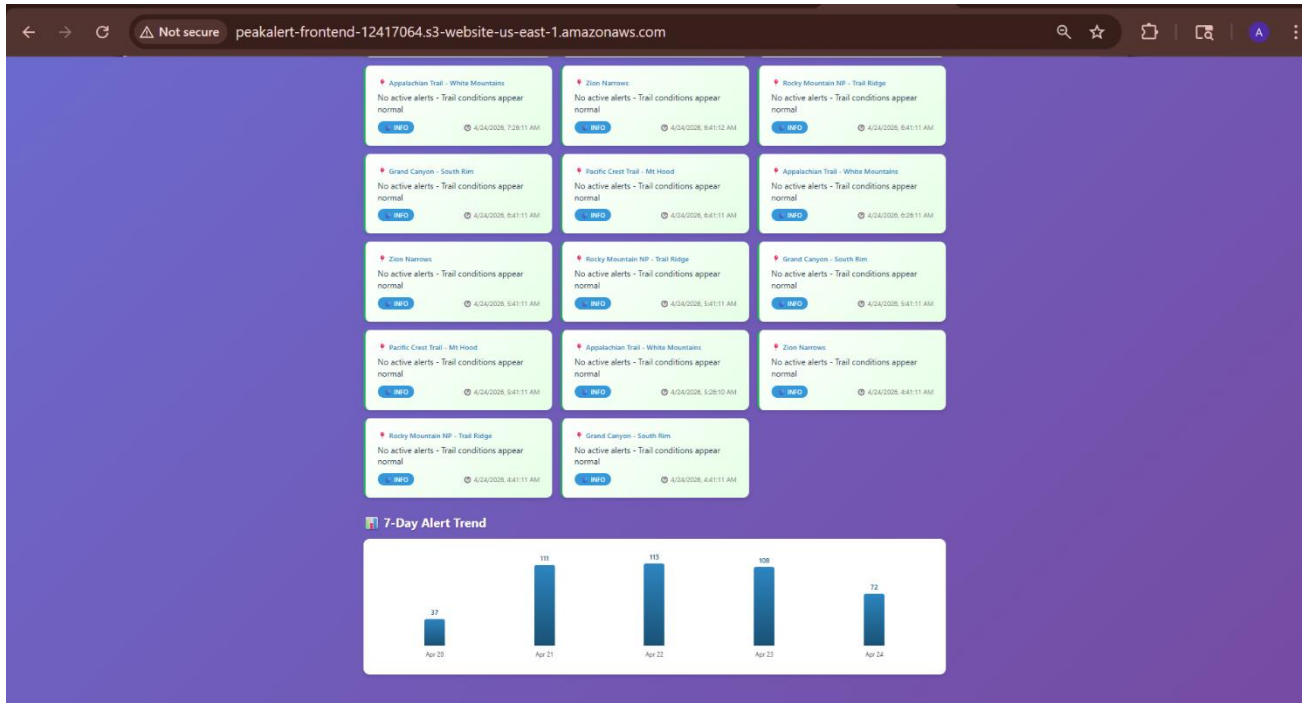


Figure 18: PeakAlert fully operational — colour-coded alert cards for all 5 trails including Appalachian Trail, Zion Narrows, Grand Canyon, Rocky Mountain NP, and Pacific Crest Trail





10. Future Enhancements

1. Auto Scaling Group for EC2

- Replace single EC2 with Launch Template plus ASG (min:1, max:3)
- CPU-based target tracking: scale out above 70%, scale in when load drops
- Makes the backend truly elastic - handles traffic spikes automatically

2. CI/CD with CodePipeline and CodeDeploy

- GitHub repo connected to CodePipeline for automated builds on every push
- CodeDeploy rolling deployment to EC2 with zero downtime - no manual SSH

3. Amazon SNS Push Notifications

- Lambda publishes to SNS when severe or extreme alerts are detected
- Users subscribe via email or SMS for instant trail hazard notifications

4. AWS Secrets Manager

- Move RDS credentials from env vars to Secrets Manager with 30-day auto-rotation
- Lambda retrieves secrets at runtime - credentials never stored in code or config

5. Multi-Region Disaster Recovery

- RDS Cross-Region Read Replica plus Route 53 failover routing
- Achieves 99.99% availability SLA - enterprise-grade resilience